Step 20 - DVFS Independence 1
Reference Note 1
Learning Objectives 1
1. What is DVFS? 1
Example 1
2. Why Is This a Problem? 1
Initial State 1
Later 1
3. Real-World Requirement 1
4. ARM Solution 1
Architecture View 1
Example 1
CPU 1
System Counter 1
5. Why Fixed Frequency Matters 1
6. Relationship to CNTFRQ_EL0 1
7. Always-On Domain 1
Benefit 1
8. What Would Happen Without Independence? 1
9. Counter Scaling vs DVFS 1
Example 1
DVFS 1
Counter Scaling 1
10. Why Shared Time Matters 1
Architecture Summary 1
Common Misconceptions 1
Misconception 1 1
Misconception 2 1
Misconception 3 1

Misconception 4 1
Key Takeaways 1
Knowledge Check 1
Q1 1
Q2 1
Q3 1
Q4 1
Q5 1
Answer Key 1

Step 20 - DVFS Independence

Reference Note

This chapter is based on concepts described in the ARM Generic Timer Architecture documentation.

One of the most important ARM timer design decisions is:

The System Counter must continue measuring time correctly even when CPU frequency changes.

This chapter explains why the Generic Timer architecture is designed to be independent of DVFS.

Learning Objectives

After completing this chapter, you should understand:

- What DVFS is
- Why CPU frequency changes occur
- Why timer accuracy would be broken if timers depended on CPU frequency
- Why the System Counter uses a fixed frequency
- Relationship between CPU clock and System Counter clock

- Always-On timer domains
- Common timing misconceptions

1. What is DVFS?

DVFS stands for:

Dynamic Voltage and Frequency Scaling

Processors use DVFS to:

Goal	Purpose
Reduce Power	Lower frequency when idle
Increase Performance	Raise frequency when busy
Reduce Heat	Lower power consumption
Improve Battery Life	Mobile devices

Example

CPU State	CPU Frequency
Idle	500 MHz
Moderate Load	1.2 GHz
Heavy Load	2.0 GHz

The processor frequency may change many times during runtime.

2. Why Is This a Problem?

Suppose a timer were based on CPU frequency.

Initial State

CPU Frequency	1 GHz
Delay	1 ms

Later

CPU Frequency	2 GHz
Same Delay Loop	0.5 ms

Timekeeping would change simply because CPU frequency changed.

This is unacceptable.

3. Real-World Requirement

Time should behave as:

Real Time	Meaning
1 ms	Always 1 ms
1 sec	Always 1 sec
10 sec	Always 10 sec

Regardless of:

- CPU Frequency
- CPU Load
- Power State

4. ARM Solution

ARM separates:

Component	Purpose

CPU Clock	Execute instructions
System Counter Clock	Measure time

These are intentionally independent.

Architecture View

Resource	Can Change?
CPU Frequency	Yes
System Counter Frequency	No

Example

CPU

State	Frequency
State 1	800 MHz
State 2	2 GHz

System Counter

State	Frequency
State 1	24 MHz
State 2	24 MHz

Timekeeping remains unchanged.

5. Why Fixed Frequency Matters

Consider:

Generate interrupt after 24,000 ticks

With:

24 MHz Counter

Result:

1 ms

Always.

Even if:

CPU = 500 MHz

or

CPU = 2.5 GHz

the timer interval remains:

1 ms

6. Relationship to CNTFRQ_EL0

Recall:

CNTFRQ_EL0

contains:

System Counter Frequency

Example:

Register	Value
CNTFRQ_EL0	24,000,000

This value is expected to remain stable because the System Counter frequency is fixed.

7. Always-On Domain

The System Counter is typically placed in an:

Always-On Power Domain

This ensures the counter continues running during:

CPU State
Active
Idle
Sleep
Low Power

Benefit

Event	Counter Running?
CPU Active	Yes
CPU Idle	Yes
CPU Sleep	Yes

Time continues advancing correctly.

8. What Would Happen Without Independence?

Suppose:

Counter Frequency == CPU Frequency

and CPU changes:

1 GHz → 2 GHz

Then:

Effect	Result
Time Measurement	Incorrect
Timeout Calculation	Incorrect
Timer Intervals	Incorrect
Scheduling	Unstable

The timer system would become unusable.

9. Counter Scaling vs DVFS

These concepts are often confused.

Feature	DVFS	Counter Scaling
Changes CPU Frequency	Yes	No
Changes Counter Clock	No	No
Changes Counter Increment	No	Yes
Purpose	Power/Performance	Time Representation

Example

DVFS

CPU
1 GHz → 2 GHz

System Counter:

24 MHz

unchanged.

Counter Scaling

System Counter Clock:

24 MHz

unchanged.

Counter behavior:

Count += 2

instead of:

Count += 1

10. Why Shared Time Matters

Imagine a multicore system:

Core	Frequency
Core 0	1 GHz
Core 1	2 GHz
Core 2	1.5 GHz

With ARM's design:

Core	System Counter
Core 0	Shared
Core 1	Shared
Core 2	Shared

All cores observe the same timeline.

Architecture Summary

Component	Purpose
CPU Clock	Execute instructions
System Counter Clock	Measure time
DVFS	Adjust CPU performance
Fixed Counter Frequency	Preserve time accuracy
Always-On Domain	Maintain continuous timekeeping

Common Misconceptions

Misconception 1

CPU clock and timer clock are the same.

False.

They are intentionally separated.

Misconception 2

A faster CPU makes timers run faster.

False.

Timer behavior is based on the System Counter.

Misconception 3

DVFS changes timer intervals.

False.

The Generic Timer architecture is designed to prevent this.

Misconception 4

CNTFRQ_EL0 tracks CPU frequency.

False.

It tracks the System Counter frequency.

Key Takeaways

Concept	Key Point
DVFS	Changes CPU performance
System Counter	Independent of DVFS
Timer Accuracy	Preserved across frequency changes
CNTFRQ_EL0	Fixed counter frequency
Always-On Domain	Counter continues during low-power states
ARM Design Goal	Stable notion of time

Knowledge Check

Q1

What does DVFS stand for?

Q2

Why can't timer accuracy depend on CPU frequency?

Q3

Does changing CPU frequency change the System Counter frequency?

Q4

Why is the System Counter typically placed in an Always-On domain?

Q5

What is stored in CNTFRQ_EL0?

Answer Key

Question	Answer
Q1	Dynamic Voltage and Frequency Scaling
Q2	Timekeeping would become inaccurate
Q3	No
Q4	To keep time advancing during low-power states
Q5	System Counter frequency